

Robust Optimization for Order Flow Imbalance Prediction

Bojue Wang (POC)

Jiayu Yang

May 2026

Abstract

This report studies short-horizon return prediction from multi-level limit order book data using robust optimization. The data consist of public LOBSTER sample files for five NASDAQ tickers with ten order-book levels, aggregated into 500ms time bins. The learning problem uses standardized order-flow and order-book features, together with an event-activity mask indicating whether a ticker has a fresh order-book update in a bin. The main method is a robust min-max training problem in which a predictive model is optimized against bounded perturbations of both input features and cross-asset graph structure. The completed pipeline compares empirical risk minimization with feature-robust and graph-robust projected gradient descent-ascent. On this one-day five-asset sample, graph-robust training gives the most consistent stability improvement: it reduces graph-perturbation MSE degradation and graph sensitivity at every tested prediction horizon. Feature-robust training is more mixed and is mainly beneficial at longer horizons.

Contents

1	Introduction	2
2	Related Work	3
3	Data and Preprocessing	3
3.1	Dataset	3
3.2	Time Binning and Splits	4
3.3	Cleaning and Unit Conventions	5
4	Feature Construction	5
4.1	Prediction Target	6
4.2	Order-Book and Order-Flow Features	6
4.3	Message-Flow and Event-Activity Features	7
4.4	Final Feature Matrix	7

5	Graph Construction	7
6	Prediction Models	8
7	Robust Optimization Formulation	9
8	Optimization Algorithm	10
9	Implementation Details	10
10	Numerical Results	11
10.1	Preprocessing Audit	11
10.2	Clean Test Performance	12
10.3	Feature-Robust Degradation	12
10.4	Graph-Robust Degradation and Sensitivity	14
10.5	Clean-Accuracy Tradeoff	16
10.6	Trading-Style Sanity Check	17
11	Analysis and Discussion	17
12	Limitations and Future Work	18
13	Conclusion	19

1 Introduction

Short-horizon return prediction from limit order book (LOB) data is a central problem in market microstructure and quantitative finance. A common signal class is *order flow imbalance* (OFI), which summarizes how updates to bid and ask queues reveal short-term supply–demand pressure. OFI features can be predictive, but their performance is often unstable because high-frequency order-book data are noisy, sparse across assets, and sensitive to changing cross-asset relationships [1, 2].

This project asks the following optimization question:

How can one train an OFI-based prediction model whose performance remains stable under structured uncertainty in both order-book features and cross-asset relationships?

The project treats this question as a robust optimization problem. Instead of training only on observed features, the model is trained against bounded adversarial perturbations of the feature matrix and the asset-interaction graph. The main hypothesis is that this min–max formulation reduces sensitivity to microstructure noise and graph misspecification, even if clean-test accuracy is not always improved.

Contributions. The report focuses on four concrete contributions:

1. a reproducible 500ms LOBSTER data pipeline for five assets and ten order-book levels;
2. a compact set of OFI, order-book imbalance, message-flow, and event-activity features;
3. a graph-linear prediction model with robust feature and graph perturbations;
4. an experimental comparison between ERM, feature-robust training, and graph-robust training using the benchmark outputs in the project repository.

2 Related Work

Order flow imbalance and price impact. OFI-based prediction is motivated by the observation that order-book events have measurable short-term price impact [1]. At high frequency, queue updates at the best bid and ask, as well as deeper limit-order-book levels, provide information about transient supply–demand pressure. This report uses multi-level OFI and order-book imbalance features as a compact representation of these short-horizon microstructure signals.

Cross-asset dependence. In multi-asset settings, order flow in one asset may be associated with future price movements in another asset. Cross-impact models study this type of dependence and motivate the use of an asset-interaction graph [2]. The present report uses a deliberately simple graph-linear model rather than a full structural market-impact model, so that the effect of the optimization method can be isolated from the complexity of the prediction architecture.

Robust optimization and adversarial training. The optimization framework is based on robust optimization and adversarial training. Classical robust optimization studies decision making under bounded uncertainty [3, 4]. Adversarial training applies a similar min–max idea to predictive models by training against worst-case perturbations [5]. In this report, the uncertainty variables represent standardized feature noise and graph misspecification.

Positioning. The project is not intended to build a production trading system. Its purpose is to test whether a robust optimization formulation gives more stable prediction and degradation behavior than ordinary empirical risk minimization on a controlled LOBSTER sample. The main contribution is therefore methodological: a small but reproducible comparison of empirical risk minimization, feature-robust training, and feature–graph-robust training for OFI-based prediction.

3 Data and Preprocessing

3.1 Dataset

The project uses public LOBSTER sample files. For each ticker, LOBSTER provides a message file recording event time, event type, order size, price, and direction, and an orderbook

file recording the resulting LOB state after each event [7]. Table 1 lists the five ten-level tickers used in the experiment.

Table 1: LOBSTER sample tickers used in the experiment.

Company	Ticker	Levels	Role in project
Amazon	AMZN	10	asset $i = 1$
Apple	AAPL	10	asset $i = 2$
Google	GOOG	10	asset $i = 3$
Intel	INTC	10	asset $i = 4$
Microsoft	MSFT	10	asset $i = 5$

Thus the main experiment has $N = 5$ assets and $H = 10$ order-book levels. All files correspond to the same trading day, June 21, 2012, over the regular trading interval 09:30:00–16:00:00. Table 2 reports the raw event-level file sizes and timestamp ranges.

Table 2: Raw event-level file sizes and timestamp ranges.

Ticker	Message rows	Orderbook rows	First timestamp	Last timestamp
AAPL	400,391	400,391	34200.0042	57599.9131
AMZN	269,748	269,748	34200.0175	57599.9594
GOOG	147,916	147,916	34200.0151	57599.8718
INTC	624,040	624,040	34200.0057	57599.9484
MSFT	668,765	668,765	34200.0140	57599.9078

3.2 Time Binning and Splits

Event-time data are aggregated into fixed 500ms bins. Let

$$\delta_{\text{bin}} = 0.5\text{s}, \quad [\kappa_0, \kappa_1) = [34200, 57600). \quad (3.1)$$

The total number of bins is

$$T = \frac{57600 - 34200}{0.5} = 46800. \quad (3.2)$$

The t -th bin is

$$\mathcal{B}_t = [34200 + 0.5(t - 1), 34200 + 0.5t), \quad t = 1, \dots, 46800. \quad (3.3)$$

For each asset and bin, order-book state variables are taken from the last available orderbook row in the bin; if no event occurs, the previous valid book state is carried forward. Message-flow variables are aggregated over messages inside the bin.

The data are split chronologically without shuffling, as shown in Table 3.

Table 3: Chronological train/validation/test split after 500ms binning.

Split	Index range	Number of bins	Clock-time interval
Train	1–32760	32760	09:30:00–14:03:00
Validation	32761–39780	7020	14:03:00–15:01:30
Test	39781–46800	7020	15:01:30–16:00:00

For a prediction horizon τ , samples whose target leaves the corresponding split are removed.

3.3 Cleaning and Unit Conventions

LOBSTER prices are recorded as dollar prices multiplied by 10,000, so all prices are divided by 10,000 before feature construction. Trading-halt indicators and invalid dummy levels are excluded from feature aggregation. Cross trades are not used in the main message-flow features. All standardization parameters are estimated on the training set only.

Preprocessing audit. The raw audit and feature-construction modules both pass their benchmark gates. The audit confirms the 500ms grid contract, the expected raw row counts, positive sample spreads, and the equality of message and order-book row counts for all tickers. The feature build confirms the output tensors X_{raw} and $\tilde{X}$ with shape $(46800, 5, 51)$, binary event masks, finite standardized features, and horizon-specific targets for $\tau \in \{1, 2, 10, 20\}$. The ticker-level audit values used in the final numerical section are reported in [table 7](#).

4 Feature Construction

The main notation used in the feature construction and robust optimization sections is summarized in [Table 4](#).

Table 4: Notation summary for data construction and robust optimization.

Symbol	Meaning	Range / dimension
t	time-bin index	$t \in \mathcal{T} = \{1, \dots, T\}$, $T = 46800$
i	asset index	$i \in \mathcal{A} = \{1, \dots, N\}$, $N = 5$
h	order-book level	$h \in \mathcal{H} = \{1, \dots, H\}$, $H = 10$
$p_{t,i}^{a,h}, p_{t,i}^{b,h}$	ask and bid prices	dollars
$q_{t,i}^{a,h}, q_{t,i}^{b,h}$	ask and bid queue sizes	shares
$m_{t,i}$	mid-price of asset i	dollars
$m_{t,i}^{\text{wm}}$	weighted-mid price of asset i	dollars
$y_{t,i}^{(\tau)}$	τ -step prediction target	log-return
$\chi_{t,i}^{\text{evt}}$	event-activity mask	$\{0, 1\}$
$x_{t,i}$	asset-level feature vector	$\mathbb{R}^d$
$\tilde{X}_t$	standardized multi-asset feature matrix	$N \times d$
G_t	cross-asset interaction matrix	$N \times N$
f_θ	prediction model	$f_\theta(\tilde{X}_t, G_t) \in \mathbb{R}^N$

Let $t \in \mathcal{T}$ index time bins, $i \in \mathcal{A}$ index assets, and $h \in \mathcal{H}$ index order-book levels. At level h , let $p_{t,i}^{a,h}, p_{t,i}^{b,h}$ denote ask and bid prices, and let $q_{t,i}^{a,h}, q_{t,i}^{b,h}$ denote ask and bid queue sizes. At the best level $h = 1$, define the ask price $a_{t,i}$, bid price $b_{t,i}$, ask depth $A_{t,i}$, and bid depth $B_{t,i}$. The mid-price, spread, and weighted-mid price are

$$m_{t,i} = \frac{a_{t,i} + b_{t,i}}{2}, \quad s_{t,i} = a_{t,i} - b_{t,i}, \quad m_{t,i}^{\text{wm}} = \frac{a_{t,i}B_{t,i} + b_{t,i}A_{t,i}}{A_{t,i} + B_{t,i}}. \quad (4.1)$$

4.1 Prediction Target

For horizon τ , the main regression target is the forward log-return

$$y_{t,i}^{(\tau)} = \log m_{t+\tau,i} - \log m_{t,i}. \quad (4.2)$$

The experiments use horizons $\tau \in \{1, 2, 10, 20\}$, corresponding to 0.5s, 1s, 5s, and 10s.

4.2 Order-Book and Order-Flow Features

The level- h order book imbalance is

$$\iota_{t,i}^h = \frac{\sum_{j=1}^h (q_{t,i}^{b,j} - q_{t,i}^{a,j})}{\sum_{j=1}^h (q_{t,i}^{b,j} + q_{t,i}^{a,j}) + \varepsilon}. \quad (4.3)$$

The dollar-weighted version replaces each queue size q by dollar depth pq .

For $t \geq 2$, the level- h bid-side OFI contribution is

$$\phi_{t,i}^{b,h} = \mathbb{1}\{p_{t,i}^{b,h} > p_{t-1,i}^{b,h}\} q_{t,i}^{b,h} + \mathbb{1}\{p_{t,i}^{b,h} = p_{t-1,i}^{b,h}\} (q_{t,i}^{b,h} - q_{t-1,i}^{b,h}) \quad (4.4)$$

$$- \mathbb{1}\{p_{t,i}^{b,h} < p_{t-1,i}^{b,h}\} q_{t-1,i}^{b,h}. \quad (4.5)$$

The ask-side contribution is defined analogously with the opposite price-move convention. The net OFI feature is

$$\phi_{t,i}^h = \phi_{t,i}^{b,h} + \phi_{t,i}^{a,h}. \quad (4.6)$$

The vector $\Phi_{t,i}^H = (\phi_{t,i}^1, \dots, \phi_{t,i}^H)^\top$ records multi-level OFI. A dollar-weighted OFI vector is also computed.

4.3 Message-Flow and Event-Activity Features

Message-flow features summarize submissions, cancellations/deletions, and executions within each 500ms bin. For each event class, the implementation records relative event price and signed event queue. The feature vector also includes traded volume and traded value.

Because many asset-bin pairs have no fresh message, the report uses an event-activity mask

$$\chi_{t,i}^{\text{evt}} = \mathbb{1}\{\text{asset } i \text{ has at least one valid message in bin } \mathcal{B}_t\}. \quad (4.7)$$

This mask is used both as an input feature and, in one model variant, as a graph-edge correlation weight.

4.4 Final Feature Matrix

The raw asset-level feature vector is

$$x_{t,i} = \left((\Phi_{t,i}^H)^\top, (\tilde{\Phi}_{t,i}^H)^\top, (I_{t,i}^H)^\top, (\tilde{I}_{t,i}^H)^\top, s_{t,i}, m_{t,i}^{\text{wm}}, \Psi_{t,i}^\top, \chi_{t,i}^{\text{evt}} \right), \quad (4.8)$$

where $\Psi_{t,i}$ denotes the message-flow block. With $H = 10$, the feature dimension is $d = 51$. Stacking assets gives $X_t \in \mathbb{R}^{N \times d}$. Each feature is standardized using training-set statistics only:

$$\tilde{x}_{t,i,k} = \frac{x_{t,i,k} - \mu_{i,k}^{\text{train}}}{\sigma_{i,k}^{\text{train}} + \varepsilon}. \quad (4.9)$$

The standardized matrix is denoted $\tilde{X}_t \in \mathbb{R}^{N \times d}$, i.e.

$$\tilde{X}_t = \begin{pmatrix} \tilde{x}_{t,1} \\ \vdots \\ \tilde{x}_{t,N} \end{pmatrix}. \quad (4.10)$$

5 Graph Construction

The cross-asset graph $G_t \in \mathbb{R}^{N \times N}$ represents recent dependence across assets. The main report uses a fixed graph estimated on the training set, with rolling graphs treated as an optional robustness check. Three graph choices are considered:

1. a return-correlation graph based on recent log returns;
2. an OFI-correlation graph based on integrated OFI signals;
3. a lagged cross-OFI graph relating past OFI in asset i to future return in asset j .

In the main implementation, the graph is symmetrized, clipped to an admissible interval, and has zero diagonal. The admissible graph set is either

$$\mathcal{G}_+ = \{A \in [0, 1]^{N \times N} : A = A^\top, \text{diag}(A) = 0\} \quad (5.1)$$

or the signed version

$$\mathcal{G}_\pm = \{A \in [-1, 1]^{N \times N} : A = A^\top, \text{diag}(A) = 0\}. \quad (5.2)$$

Mask-aware graph. To down-weight stale cross-asset propagation, define a rolling coactivity weight

$$\bar{A}_{t,ij}^{\text{evt}} = \frac{1}{|\mathcal{W}_t|} \sum_{u \in \mathcal{W}_t} \chi_{u,i}^{\text{evt}} \chi_{u,j}^{\text{evt}}. \quad (5.3)$$

The mask-aware graph is

$$G_{t,ij}^{\text{mask}} = \bar{A}_{t,ij}^{\text{evt}} G_{t,ij}, \quad G_{t,ii}^{\text{mask}} = 0. \quad (5.4)$$

Graph diagnostic. Table 5 summarizes the graph matrices produced by the training-period graph construction module. The return-correlation graph has the largest edge scale among the tested graphs. The validation-selected baseline therefore uses the unsigned return graph at horizons $\tau \in \{2, 10, 20\}$, while the shortest horizon selects the single-asset lasso baseline.

Table 5: Training-period graph summaries from the Step 03 benchmark.

Graph	Min	Max	Mean absolute edge	Nonzero edges
return_signed	0	0.171938	0.091919	20
return_abs	0	0.171938	0.091919	20
return_abs_mask	0	0.101536	0.045101	20
ofi_signed	-0.007447	0.028413	0.008474	20
ofi_abs	0	0.028413	0.008474	20
cross_ofi_signed	-0.000994	0.013188	0.004478	20
cross_ofi_abs	0	0.013188	0.004478	20

6 Prediction Models

The main model family is deliberately simple so that the optimization effect is interpretable. The graph-linear predictor is

$$\hat{Y}_t^{(\tau)} = \tilde{X}_t w_0 + G_t^\# \tilde{X}_t w_1 + \beta_0 \mathbf{1}_N, \quad (6.1)$$

where $w_0, w_1 \in \mathbb{R}^d$, $\beta_0 \in \mathbb{R}$, and $G_t^\#$ is either G_t or G_t^{mask} . The first term uses each asset’s own features, while the second term propagates neighboring assets’ features through the graph.

The report compares the variants in Table 6.

Table 6: Prediction model variants and robustness settings.

Model	Features	Graph	Robustness
Ridge/Lasso	standardized features	none	no
Graph-linear ERM	full $\widetilde{X}_t$	G_t	no
Mask-aware ERM	full $\widetilde{X}_t$ plus mask	G_t^{mask}	no
Feature-robust	full $\widetilde{X}_t$ plus mask	$G_t^\sharp$	$\rho_X > 0, \rho_G = 0$
Feature-graph robust	full $\widetilde{X}_t$ plus mask	$G_t^\sharp$	$\rho_X > 0, \rho_G > 0$

7 Robust Optimization Formulation

Let the training data be

$$\mathcal{D}_\tau = \{(\widetilde{X}_t, G_t, Y_t^{(\tau)}) : t \in \mathcal{T}_{\tau, \text{train}}\}. \quad (7.1)$$

For regression, the loss is

$$\ell(\widehat{Y}, Y) = \frac{1}{N} \|\widehat{Y} - Y\|_2^2. \quad (7.2)$$

The ordinary empirical risk minimization objective is

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}_{\tau, \text{train}}|} \sum_{t \in \mathcal{T}_{\tau, \text{train}}} \ell(f_\theta(\widetilde{X}_t, G_t), Y_t^{(\tau)}) + \lambda_\theta \Omega(\theta). \quad (7.3)$$

Robust training introduces a feature perturbation $U_t \in \mathbb{R}^{N \times d}$ and a graph perturbation $\Delta_t \in \mathbb{R}^{N \times N}$. The feature uncertainty set is

$$\mathcal{U}_X = \{U : \|U\|_F \leq \rho_X, \|U_{i,\cdot}\|_2 \leq \rho_{X,i} \text{ for all } i\}. \quad (7.4)$$

The graph uncertainty set is

$$\mathcal{U}_G(G_t; \mathcal{G}_*) = \{\Delta : \|\Delta\|_F \leq \rho_G, G_t + \Delta \in \mathcal{G}_*\}, \quad (7.5)$$

where $\mathcal{G}_*$ is either $\mathcal{G}_+$ or $\mathcal{G}_\pm$.

The main robust objective is

$$\min_{\theta \in \Theta} \frac{1}{|\mathcal{T}_{\tau, \text{train}}|} \sum_{t \in \mathcal{T}_{\tau, \text{train}}} \max_{U_t \in \mathcal{U}_X, \Delta_t \in \mathcal{U}_G(G_t; \mathcal{G}_*)} \ell(f_\theta(\widetilde{X}_t + U_t, G_t + \Delta_t), Y_t^{(\tau)}) + \lambda_\theta \Omega(\theta). \quad (7.6)$$

This is the central optimization problem of the report. The inner maximization searches for bounded feature and graph perturbations that increase prediction loss; the outer minimization chooses parameters that remain stable under these perturbations.

First-order interpretation. For small perturbation radii, robust training approximately penalizes sensitivity:

$$\ell_t^{\text{rob}} \approx \ell_t + \rho_X \|\nabla_{\widetilde{X}_t} \ell_t\|_F + \rho_G \|\nabla_{G_t} \ell_t\|_F. \quad (7.7)$$

Thus the robust objective acts like a gradient regularizer against feature noise and graph misspecification.

8 Optimization Algorithm

The robust problem is solved by projected gradient descent-ascent (PGDA). For each mini-batch, the adversarial variables are initialized at zero and updated by projected gradient ascent:

$$U_t^{r+1} = \Pi_{\mathcal{U}_X} (U_t^r + \eta_U \nabla_{U_t} \ell_t), \quad (8.1)$$

$$\Delta_t^{r+1} = \Pi_{\mathcal{U}_G(G_t; \mathcal{G}_*)} (\Delta_t^r + \eta_G \nabla_{\Delta_t} \ell_t). \quad (8.2)$$

After R_{adv} inner steps, the model parameter is updated by gradient descent:

$$\theta^{k+1} = \Pi_{\Theta} (\theta^k - \eta_{\theta} \nabla_{\theta} L_{\mathcal{B}_k}(\theta^k, U^{R_{\text{adv}}}, \Delta^{R_{\text{adv}}})) . \quad (8.3)$$

Hyperparameters. The initial grid is

$$\rho_X \in \{0, 0.05, 0.10, 0.20\}, \quad \rho_G \in \{0, 0.05, 0.10\}, \quad R_{\text{adv}} \in \{1, 3, 5\}. \quad (8.4)$$

The validation set selects the perturbation radii, regularization strength, step sizes, and graph choice.

9 Implementation Details

The submitted code repository is

<https://github.com/bojuewang/robust-ofi-lobster>.

The submitted repository state is tagged as phase2-submission. The repository is organized as a benchmark-gated pipeline: each step writes a markdown or JSON report to `reports/benchmarks/` and writes a PASS marker to `.pipeline_state/`. Raw LOBSTER zip files are intentionally not committed.

Software environment. The repository uses Python 3 with `numpy>=1.24`, `pandas>=2.0`, `scikit-learn>=1.3`, and `matplotlib>=3.7`. The experiments are designed to run from a local virtual environment created by

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Data dependencies. The five LOBSTER sample zip files are expected under `data/raw/`: AMZN, AAPL, GOOG, INTC, and MSFT for June 21, 2012 with ten book levels. The repository ignores `data/raw/*.zip`, raw CSV files, and generated processed arrays so that large or proprietary local data are not committed.

Pipeline scripts. The full pipeline through the final summary is executed by

```
bash scripts/run_pipeline_to_step06.sh
```

which runs the following modules in order:

1. `step01_lobster_raw_audit_clean.sh`: raw LOBSTER audit and cleaning checks;
2. `step02_feature_build_500ms.sh`: 500ms feature tensor construction;
3. `step03_graph_build_and_baseline.sh`: graph construction and ERM baselines;
4. `step04_feature_robust_pgda.sh`: initial feature-robust PGDA benchmark;
5. `step04b_feature_robust_calibrated_eval.sh`: calibrated feature-adversarial evaluation;
6. `step05_graph_robust_pgda.sh`: graph-robust PGDA benchmark;
7. `step06_feature_graph_robust_summary.sh`: final ablation table and report-ready figures.

Validation protocol and determinism. Hyperparameters are selected on the chronological validation split. Stochastic training modules use fixed NumPy random seeds. The initial feature-robust PGDA module uses seed 7; the calibrated feature-adversarial evaluation uses base seed 11 with horizon/radius/inner-step offsets, namely $11 + \tau + \lfloor 1000\rho_X \rfloor + R_{\text{adv}}$; and the graph-robust PGDA module uses seed 23. The repository notes that small floating-point differences may occur across BLAS/LAPACK implementations, so benchmark gates use tolerances rather than exact bitwise equality. All final checks in the Step 06 summary pass for horizons $\tau \in \{1, 2, 10, 20\}$.

10 Numerical Results

The numerical results are taken from the benchmark artifacts in the project repository. The final tables aggregate outputs from Step 01–Step 06. All reported MSE values are out-of-sample test quantities unless explicitly marked as validation quantities.

10.1 Preprocessing Audit

[Table 7](#) reports the raw audit and feature-construction summary. The pipeline verifies 46,800 total 500ms bins and 46,290 bins with at least one event in at least one asset. The active-bin column counts bins in which the individual ticker has at least one valid message. Carry-forward bins are the remaining ticker-specific bins in which the most recent valid book state is carried forward.

Table 7: Preprocessing audit by ticker. Raw and valid row counts agree in the benchmarked continuous-trading sample.

Ticker	Raw rows	Valid rows	Active bins	Carry-forward bins	Dropped rows
AMZN	269,748	269,748	29,801	16,999	0
AAPL	400,391	400,391	34,800	12,000	0
GOOG	147,916	147,916	22,678	24,122	0
INTC	624,040	624,040	36,086	10,714	0
MSFT	668,765	668,765	37,005	9,795	0

The feature module also reports zero invalid deep-level cells for all five tickers. The standardized training features have maximum absolute mean error 1.26942×10^{-5} and maximum absolute standard-deviation error 5.55388×10^{-8} after standardization.

10.2 Clean Test Performance

The main clean-test metrics are mean squared error and directional accuracy:

$$\text{MSE}_{\text{test}} = \frac{1}{|\mathcal{T}_{\tau, \text{test}}|N} \sum_{t \in \mathcal{T}_{\tau, \text{test}}} \|\hat{Y}_t^{(\tau)} - Y_t^{(\tau)}\|_2^2, \quad (10.1)$$

$$\text{DA} = \frac{1}{|\mathcal{T}_{\tau, \text{test}}|N} \sum_{t \in \mathcal{T}_{\tau, \text{test}}} \sum_{i=1}^N \mathbb{1}\{\text{sign}(\hat{y}_{t,i}^{(\tau)}) = \text{sign}(y_{t,i}^{(\tau)})\}. \quad (10.2)$$

Table 8 reports the validation-selected ERM baseline for each horizon. The shortest horizon selects the single-asset lasso model, while horizons 2, 10, and 20 select the graph-linear ridge model with the unsigned return-correlation graph.

Table 8: Best validation-selected ERM baseline by prediction horizon.

τ	Model	Graph	Validation MSE	Test MSE	Test MAE	Directional accuracy
1	lasso_own	none	1.50086×10^{-9}	2.92900×10^{-9}	1.72549×10^{-5}	0.602310
2	graph_linear_ridge	return_abs	3.06471×10^{-9}	5.93836×10^{-9}	3.30375×10^{-5}	0.595855
10	graph_linear_ridge	return_abs	1.65465×10^{-8}	3.17499×10^{-8}	1.07269×10^{-4}	0.575196
20	graph_linear_ridge	return_abs	3.51101×10^{-8}	6.60782×10^{-8}	1.72053×10^{-4}	0.551388

Directional accuracy is above 50% at all four horizons, but it declines as the prediction horizon lengthens. This is consistent with the intuition that longer-horizon returns contain less directly recoverable short-term queue-pressure information.

10.3 Feature-Robust Degradation

Step 04b recalibrates feature-adversarial evaluation by using normalized steepest-ascent directions on the standardized feature tensor. This avoids the zero-degradation artifact observed when raw gradients are too small. Table 9 compares the ERM reference with the validation-selected feature-robust model under standardized feature perturbation radius $\rho = 0.1$.

Table 9: Feature-robust comparison under standardized feature perturbation radius $\rho = 0.1$. Positive degradation improvement means the robust model degrades less than ERM.

τ	Robust choice	ERM clean MSE	Robust clean MSE	ERM degradation	Robust degradation	Improvement	Verdict
1	$\rho = 0.05, R = 3$	2.97538×10^{-9}	3.03067×10^{-9}	3.87839×10^{-9}	4.11762×10^{-9}	-2.39226×10^{-10}	mixed
2	$\rho = 0.05, R = 1$	5.94860×10^{-9}	6.22436×10^{-9}	1.69575×10^{-8}	1.88243×10^{-8}	-1.86677×10^{-9}	mixed
10	$\rho = 0.20, R = 3$	3.22132×10^{-8}	1.28169×10^{-7}	4.81330×10^{-7}	4.11917×10^{-7}	6.94136×10^{-8}	improves
20	$\rho = 0.20, R = 1$	6.62759×10^{-8}	4.26738×10^{-7}	1.66960×10^{-6}	1.59678×10^{-6}	7.28205×10^{-8}	improves

Feature-robust training therefore does not uniformly improve robustness. It worsens degradation at $\tau = 1$ and $\tau = 2$, but improves degradation at $\tau = 10$ and $\tau = 20$. The longer-horizon gains come with a visible clean-MSE tradeoff, especially at $\tau = 20$.

10.4 Graph-Robust Degradation and Sensitivity

Table 10 gives the graph-robust comparison under graph perturbation radius $\rho = 0.1$. This is the strongest empirical result in the report: graph-robust training reduces MSE degradation and graph sensitivity at every tested horizon.

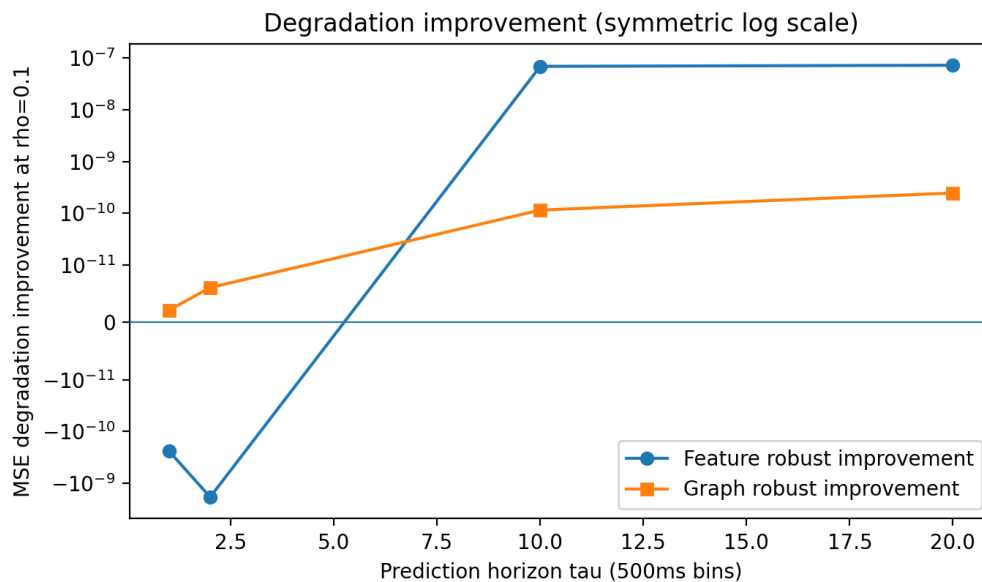


Figure 1: MSE degradation improvement at perturbation radius $\rho = 0.1$. Feature robustness is mixed; graph robustness is positive at all horizons.

Table 10: Graph-robust comparison under graph perturbation radius $\rho = 0.1$. Positive values indicate improvement relative to the ERM reference.

τ	Robust choice	ERM clean MSE	Robust clean MSE	ERM degradation	Robust degradation	Degradation improvement	S_G improvement
1	$\rho = 0.10, R = 1$	2.97538×10^{-9}	2.98385×10^{-9}	8.29492×10^{-12}	6.19771×10^{-12}	2.09721×10^{-12}	1.53701×10^{-11}
2	$\rho = 0.20, R = 1$	5.94860×10^{-9}	5.92263×10^{-9}	1.14020×10^{-11}	5.35705×10^{-12}	6.04494×10^{-12}	6.86322×10^{-11}
10	$\rho = 0.20, R = 3$	3.22132×10^{-8}	3.17908×10^{-8}	2.20997×10^{-10}	1.06201×10^{-10}	1.14795×10^{-10}	9.38278×10^{-10}
20	$\rho = 0.20, R = 3$	6.62759×10^{-8}	6.56749×10^{-8}	6.70944×10^{-10}	4.25274×10^{-10}	2.45670×10^{-10}	2.52479×10^{-9}

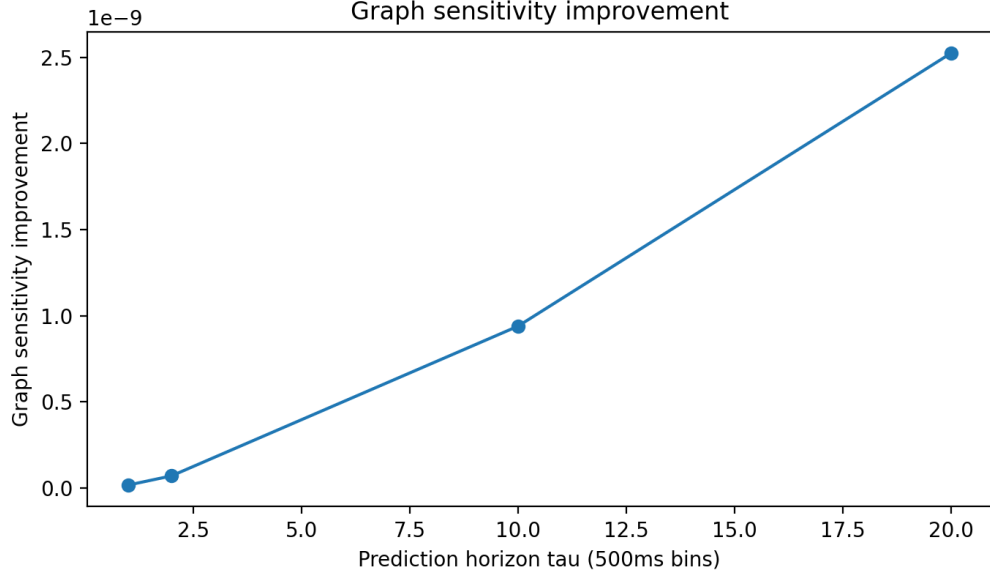


Figure 2: Graph sensitivity improvement $S_G(\theta_{\text{ERM}}) - S_G(\theta_{\text{robust}})$ by horizon. All values are positive.

10.5 Clean-Accuracy Tradeoff

Robust optimization is not primarily a clean-accuracy booster in this experiment. [Figure 3](#) compares clean MSE for the ERM reference, feature-robust model, and graph-robust model. Feature-robust training can substantially increase clean MSE at the longer horizons, while graph-robust training remains close to the ERM reference and slightly improves clean MSE at $\tau = 2, 10, 20$.

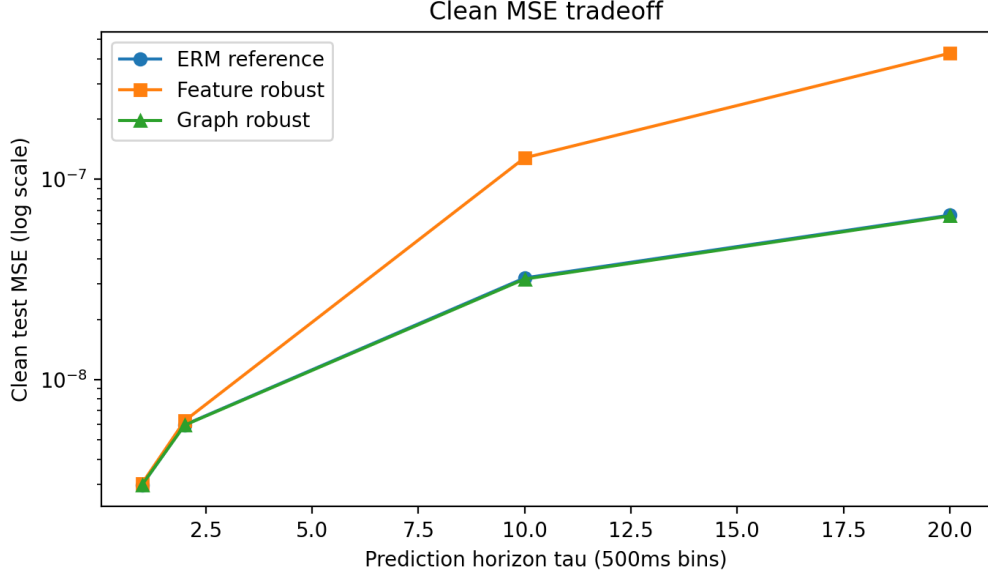


Figure 3: Clean test MSE tradeoff across ERM, feature-robust, and graph-robust models.

10.6 Trading-Style Sanity Check

As a diagnostic rather than a production backtest, define normalized long-short weights

$$w_{t,i} = \frac{\hat{y}_{t,i}^{(\tau)}}{\sum_{j=1}^N |\hat{y}_{t,j}^{(\tau)}| + \varepsilon}. \quad (10.3)$$

The Step 03 benchmark reports a no-transaction-cost Sharpe proxy for the validation-selected ERM baselines. These values are reported in [table 11](#). They are useful only as a sign-and-scale sanity check; they do not model execution, latency, slippage, borrow constraints, or market impact.

Table 11: Trading-style sanity check for validation-selected ERM baselines.

τ	Model	Graph	Sharpe proxy, no transaction cost
1	lasso_own	none	0.225876
2	graph_linear_ridge	return_abs	0.269888
10	graph_linear_ridge	return_abs	0.347969
20	graph_linear_ridge	return_abs	0.313968

11 Analysis and Discussion

The experiment supports the robust-optimization framing, but with an important distinction between feature uncertainty and graph uncertainty. The ERM baselines already give finite test MSE and directional accuracy above 50% at all horizons. Therefore, the central question

is not whether the model can fit a short-horizon signal, but whether robust training improves stability when the inputs or graph are perturbed.

Clean prediction. The best clean baseline at $\tau = 1$ is a lasso model using only own-asset standardized features. For $\tau \in \{2, 10, 20\}$, the validation-selected baseline is graph-linear ridge with the unsigned return-correlation graph. This suggests that cross-asset information becomes more useful as the horizon extends beyond the immediate 500ms bin. Directional accuracy declines from 0.602 at $\tau = 1$ to 0.551 at $\tau = 20$, so the longer-horizon prediction task is harder despite having larger raw returns.

Feature robustness. Feature-robust training is mixed. At $\tau = 1$ and $\tau = 2$, the robust models degrade more under the calibrated feature perturbation than the ERM reference. At $\tau = 10$ and $\tau = 20$, they reduce degradation, but this improvement comes with a significant clean-MSE cost. This indicates that feature-level adversarial training is sensitive to perturbation calibration and may over-regularize the graph-linear model when the horizon is short.

Graph robustness. Graph-robust training gives the clearest improvement. At every tested horizon, it reduces graph-perturbation MSE degradation and lowers graph sensitivity. The clean-MSE cost is also small: it is slightly negative at $\tau = 1$ and slightly positive at $\tau = 2, 10, 20$. This makes graph robustness the strongest conclusion of the project. In this sample, the main instability appears to come less from standardized feature noise itself and more from uncertainty in the estimated cross-asset interaction graph.

Event-activity mask. The mask-aware variants are computationally feasible and help encode whether a ticker has a fresh event in the current or recent bin. However, the final validation-selected clean baselines do not consistently choose the mask-aware graph. The mask is therefore best interpreted as a useful engineering safeguard against stale carry-forward states rather than as the dominant source of performance improvement in the current one-day sample.

Accuracy-robustness tradeoff. The final result is a stability result rather than a nominal accuracy result. Robust optimization does not uniformly improve clean prediction error. Instead, graph-robust PGDA improves degradation behavior and sensitivity under structured graph perturbations. Feature-robust PGDA can help at longer horizons, but it introduces a stronger clean-accuracy tradeoff.

12 Limitations and Future Work

The current experiment has several limitations. First, the public LOBSTER sample covers only one trading day and five tickers, so the results should be interpreted as a controlled numerical study rather than a broad empirical claim. Second, the graph-linear model is intentionally simple; nonlinear models may capture richer interactions but would make the

optimization comparison less transparent. Third, the trading-style PnL is only a sanity check and does not include realistic execution, latency, slippage, borrow constraints, or market impact. Finally, rolling graph estimation may be noisy because the sample is short and the asset universe is small.

Future work could extend the experiment to more days and assets, compare fixed and rolling graphs more systematically, introduce nonlinear graph neural models, or replace bounded perturbation sets with distributionally robust uncertainty sets.

13 Conclusion

This report formulated short-horizon OFI-based prediction as a robust optimization problem. The implemented pipeline combines multi-level LOB features, event-activity masks, cross-asset graphs, and min-max training with bounded feature and graph perturbations. The numerical evidence shows that graph-robust PGDA is the most reliable robustness mechanism in the current experiment: it reduces graph-perturbation MSE degradation and graph sensitivity at all four tested horizons. Feature-robust PGDA is more mixed, failing to improve degradation at the two shortest horizons but improving degradation at $\tau = 10$ and $\tau = 20$ with a noticeable clean-MSE tradeoff.

The main conclusion is therefore not that robust optimization automatically improves nominal forecasting accuracy. Rather, it provides a structured way to control degradation under plausible perturbations. For this five-asset one-day LOBSTER sample, robustness to cross-asset graph misspecification is more stable and interpretable than feature-level adversarial robustness.

References

- [1] R. Cont, A. Kukanov, and S. Stoikov, “The price impact of order book events,” *Journal of Financial Econometrics*, 2014.
- [2] R. Cont, M. Cucuringu, and C. Zhang, “Cross-impact of order flow imbalance in equity markets,” 2023.
- [3] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*, Princeton University Press, 2009.
- [4] D. Bertsimas, D. Brown, and C. Caramanis, “Theory and applications of robust optimization,” *SIAM Review*, 2011.
- [5] A. Madry et al., “Towards deep learning models resistant to adversarial attacks,” *ICLR*, 2018.
- [6] LOBSTER, “Sample files,” <https://data.lobsterdata.com/info/DataSamples.php>.
- [7] LOBSTER, “Output structure,” <https://data.lobsterdata.com/info/DataStructure.php>.